



NextStep — Référence Complète du Projet

NextStep est une plateforme d'accompagnement IA pour les étudiants et jeunes diplômés à la recherche de stages (PFA/PFE) ou d'un premier emploi. Elle automatise le processus complet : **Analyse d'offre** → **Matching** → **Génération CV** → **Email** → **Suivi** → **Relance**.



Table des matières

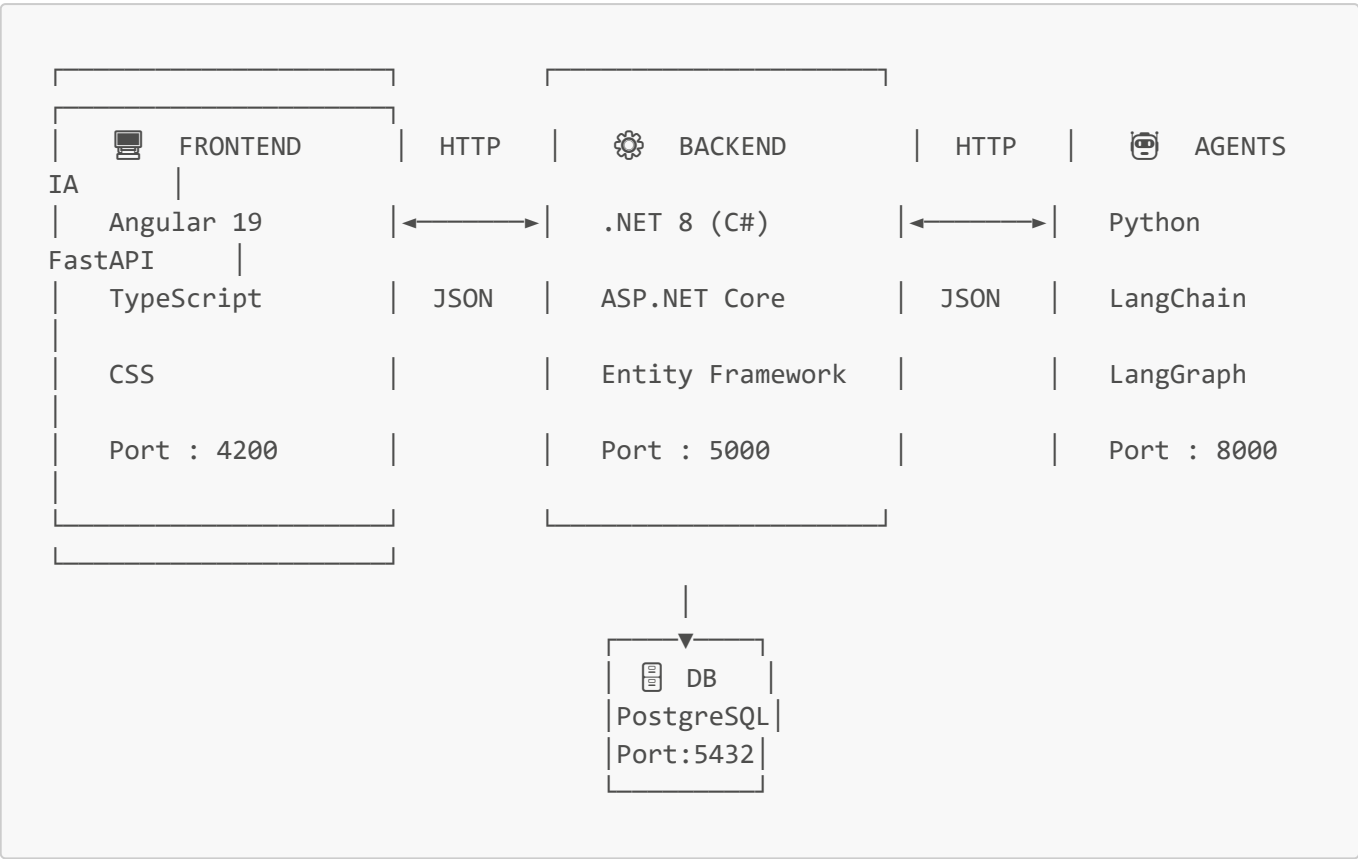
1. [Présentation du projet](#)
2. [Stack technologique complète](#)
3. [Architecture globale \(Monolithe Modulaire\)](#)
4. [Pipeline Multi-Agent Optimisé \(LangGraph\)](#)
5. [Les 11 fonctionnalités détaillées](#)
6. [Structure des fichiers](#)
7. [Communication entre services](#)
8. [Base de données — Schéma](#)
9. [Endpoints API complets](#)
10. [Infrastructure & Docker](#)
11. [Sécurité & Authentification](#)
12. [Variables d'environnement](#)
13. [Équipe & Répartition](#)

1. Présentation du projet

Ce que fait NextStep concrètement

1. L'utilisateur COLLE une offre d'emploi et CHOISIT un template de CV
2. L'IA ANALYSE l'offre → extrait compétences, mots-clés, type de poste
3. L'IA COMPARE le profil utilisateur avec l'offre → score de matching
4. Le système GÉNÈRE un CV PDF personnalisé avec un SCORE ATS
5. L'IA RÉDIGE un email de candidature personnalisé
6. L'utilisateur VALIDE et ENVOIE → email + CV en pièce jointe
7. Le système SURVEILLE la boîte mail pour détecter les réponses
8. Après 7 JOURS sans réponse → RELANCE AUTOMATIQUE
9. Un CHATBOT IA prépare l'utilisateur à l'entretien

Les 3 briques du système



Architecture choisie : Monolithe Modulaire Pas d'event-driven, pas de Kafka, pas de microservices. Communication simple et directe : **Angular** → **.NET** → **Python** via HTTP/JSON.

2. Stack technologique complète

Frontend

Technologie	Version	Rôle	Détails
Angular	19	Framework SPA	Routing, lazy-loading, standalone components
TypeScript	5.x	Langage typé	Sécurité du code, interfaces typées
CSS	—	Styles	Responsive design, variables CSS
RxJS	7.x	Réactivité	Observables, Subjects, gestion async
keycloak-js	24.x	Auth SDK	Intégration SSO, gestion tokens PKCE
@microsoft/signalr	—	WebSocket client	Réception notifications temps réel

Backend (.NET)

Technologie	Version	Rôle	Détails
ASP.NET Core	8.0	API REST	Middleware, DI, Controllers
Entity Framework Core	8.0	ORM	Mapping C# → PostgreSQL, Migrations

Technologie	Version	Rôle	Détails
QuestPDF	latest	Génération PDF	Layout engine C#, templates pré-conçus
MailKit	latest	SMTP	Envoi emails, pièces jointes PDF
SignalR	built-in	WebSocket serveur	Notifications push temps réel
Hangfire	latest	Background jobs	Tâches planifiées (relance 7j), dashboard

 Agents IA (Python)

Technologie	Version	Rôle	Détails
Python	3.12	Langage	Runtime principal des agents
FastAPI	0.115+	API REST	Endpoints rapides, validation auto
LangChain	0.3+	Framework IA	Orchestration prompts LLM
LangGraph	0.2+	Multi-Agent	Graphe d'agents, state partagé
Pydantic	2.x	Validation	Schémas JSON, contrats d'API
pydantic-settings	2.x	Config	Variables d'environnement typées
SQLAlchemy	2.0+	ORM async	Requêtes PostgreSQL depuis Python
asyncpg	0.29+	Driver DB	PostgreSQL async natif
httpx	0.27+	Client HTTP	Appels async vers .NET
imaplib	stdlib	Email monitor	Polling IMAP (surveillance boîte mail)
unidecode	1.3+	Normalisation	Suppression accents pour matching

 Infrastructure

Technologie	Version	Rôle	Port
PostgreSQL + pgvector	15	BDD relationnelle + Vector Search (RAG)	5432
Keycloak	24	Serveur d'identité SSO	8080
Redis	7 (alpine)	Cache + Hangfire jobs	6379
Docker Compose	v2	Orchestration conteneurs	—

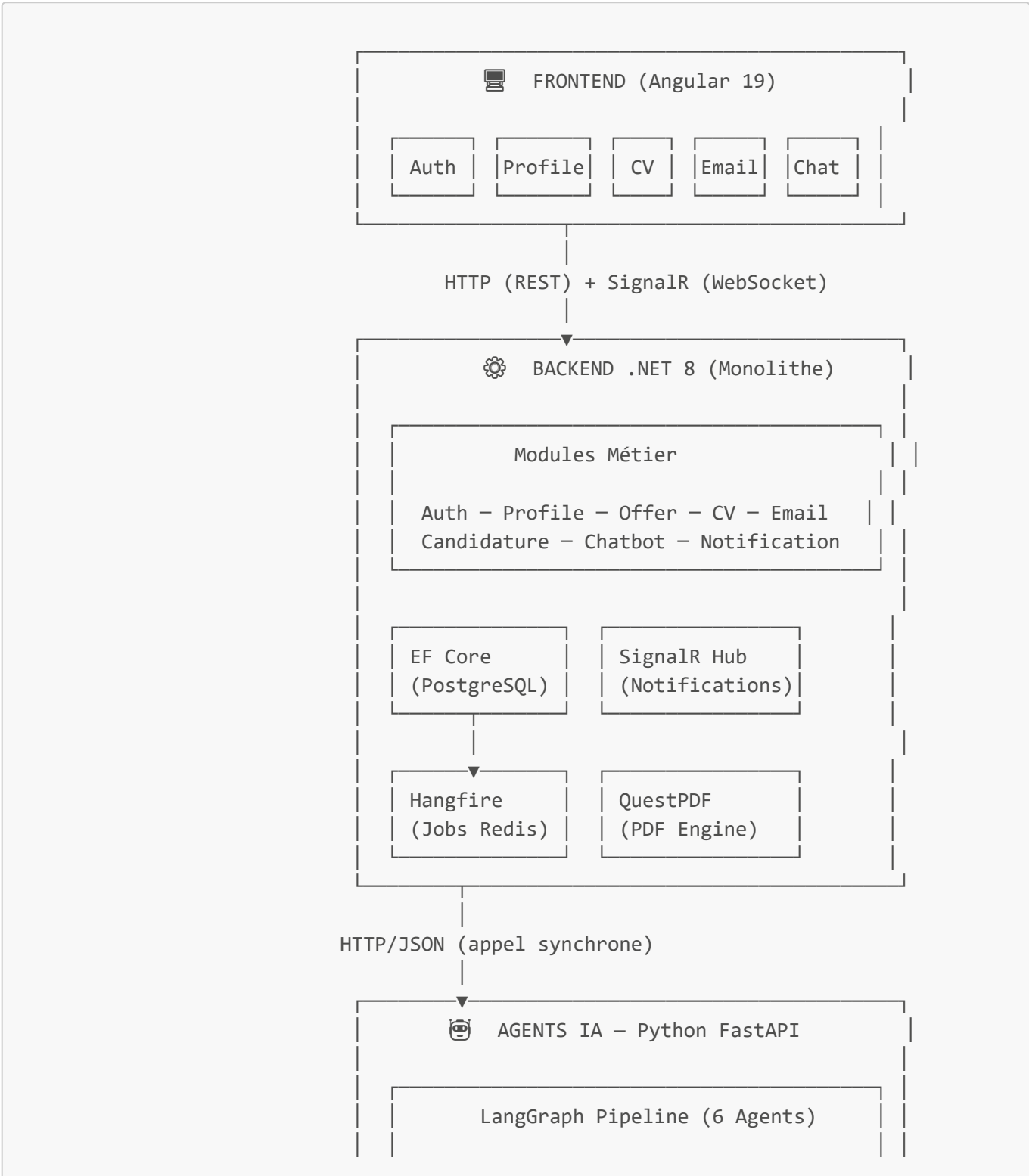
3. Architecture globale (Monolithe Modulaire)

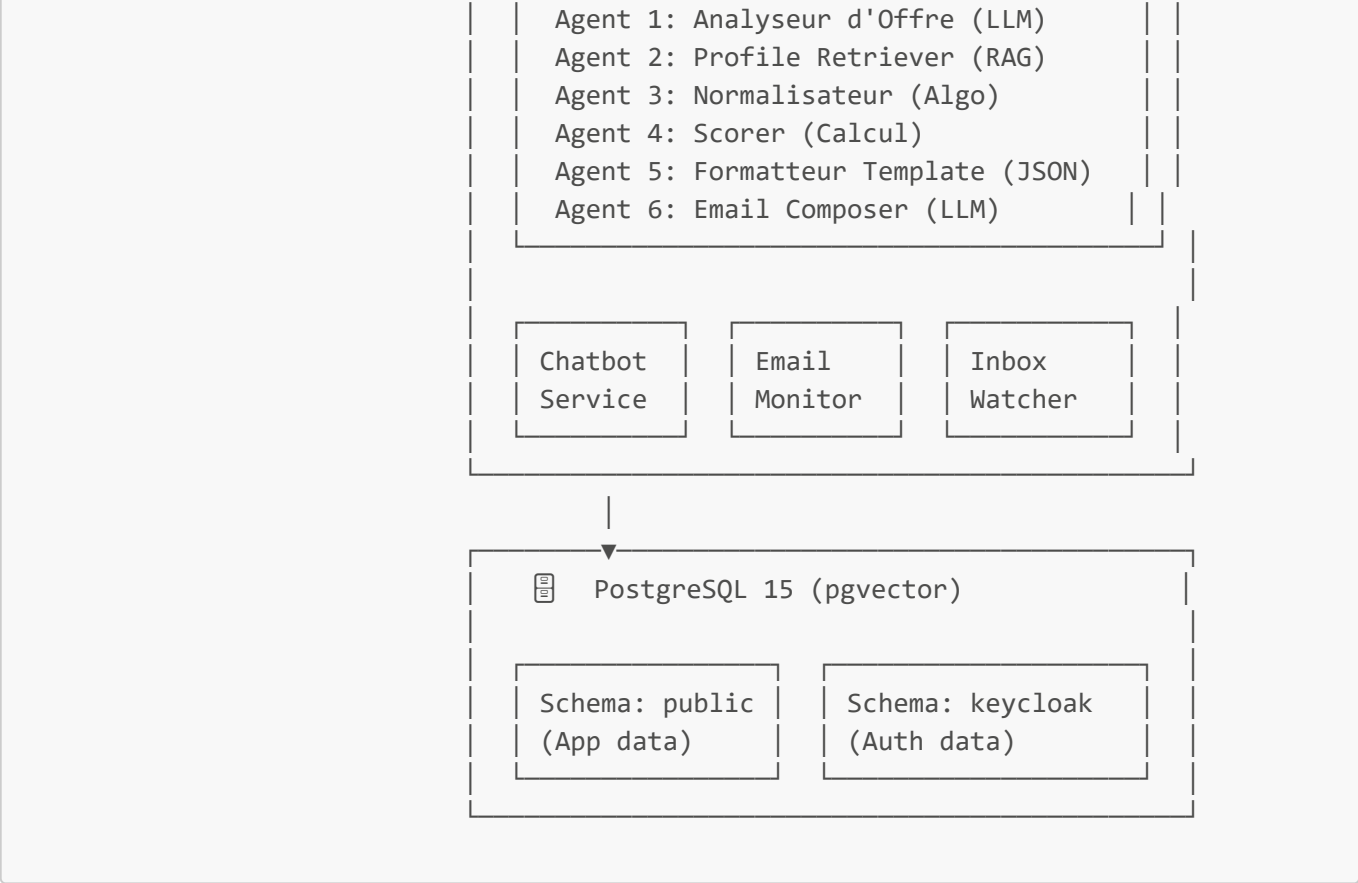
Pourquoi un Monolithe Modulaire ?

Critère	Monolithe Modulaire ☒	Microservices ☐
Complexité	Simple à développer et débbugger	Trop complexe pour un projet académique

Critère	Monolithe Modulaire 	Microservices 
Déploiement	1 seul docker-compose up	Orchestration Kubernetes nécessaire
Communication	HTTP direct, pas de broker	Kafka/RabbitMQ à gérer
Équipe	5 personnes, parfait	Nécessite +10 personnes
Performance	Appels in-process rapides	Latence réseau entre services
Debug	Stack trace unique	Tracing distribué (Jaeger, Zipkin)

Schéma d'architecture complet





Flux de communication simplifié

1. Utilisateur → Angular	(UI interaction)
2. Angular → .NET	(HTTP REST + Bearer JWT)
3. .NET → Python	(HTTP/JSON synchrone)
4. Python → PostgreSQL	(SQLAlchemy async pour RAG)
5. Python → .NET	(Retour résultat JSON)
6. .NET → Angular	(Réponse HTTP + SignalR push)

Règle fondamentale : Pas de message broker (Kafka/RabbitMQ). Chaque appel est **synchrone HTTP/JSON**. Le backend .NET est le **chef d'orchestre**. Il reçoit les requêtes du frontend, appelle les agents Python si nécessaire, et retourne la réponse.

Pattern Clean Architecture par module

.NET — Chaque module suit le même pattern :

Module/	
— Controllers/	← Reçoit HTTP, valide, délègue
— Services/	← Logique métier (ne connaît pas EF Core)
— Repositories/	← Accès DB (EF Core)
— DTOs/	← Contrats JSON entrée/sortie
— Models/	← Entités (tables PostgreSQL)

Python — Chaque service suit le même pattern :

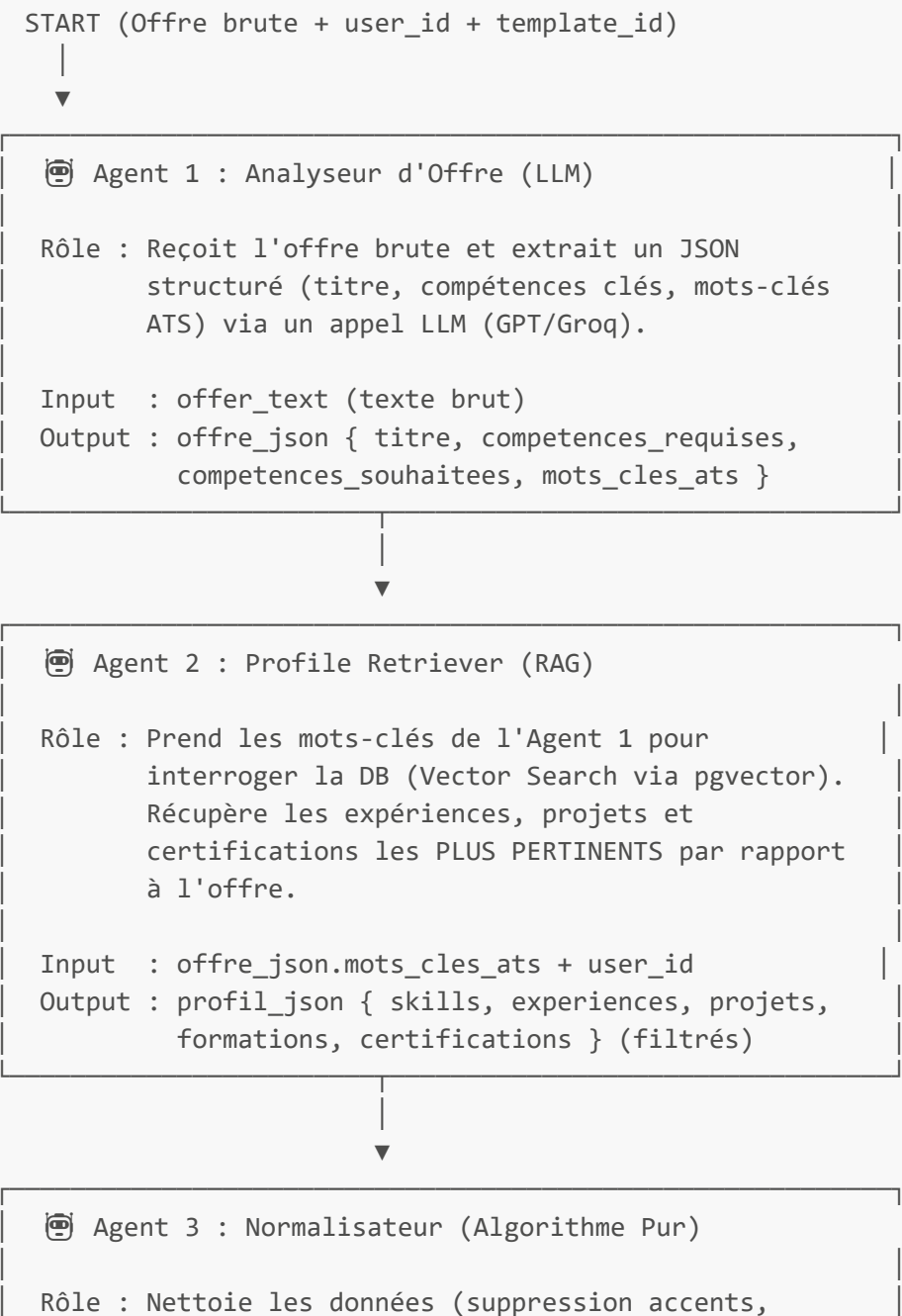
```
service/  
├─ router.py      ← Endpoint FastAPI  
├─ service.py    ← Logique métier  
└─ schemas.py    ← Modèles Pydantic (contrats JSON)
```

4. 🤖 Pipeline Multi-Agent Optimisé (LangGraph)

Philosophie du Pipeline

L'ordre d'exécution est conçu pour **maximiser la pertinence des données** avant la génération finale. Chaque agent a un **rôle unique** et bien défini.

Les 6 Agents + Agent de Notification



synonymes, mise en minuscules) pour garantir que le matching ultérieur soit précis.

PAS de LLM – Algorithme déterministe (unidecode).

Input : offre_json + profil_json

Output : données normalisées



 Agent 4 : Scorer (Calcul Mathématique)

Rôle : Calcule le score de matching ET le score ATS.

Utilise des pondérations mathématiques :

- Mot-clé dans le TITRE → +5 pts
- Mot-clé dans le RÉSUMÉ → +3 pts
- Mot-clé dans COMPÉTENCES → +2 pts
- Mot-clé dans EXPÉRIENCES → +1 pt

PAS de LLM – PAS de scikit-learn – Calcul pur.

Input : données normalisées

Output : match_result { score_matching, score_atc,
competences_matchees, gaps, recommandations }



 Agent 5 : Formateur de Template (JSON Structuré)

Rôle : Organise TOUTES les données (profil récupéré + scores) dans le schéma JSON attendu par le moteur PDF (.NET QuestPDF).

PAS de LLM – Transformation de structure pure.

Input : profil filtré + scores + template_id

Output : cv_data_json (prêt pour QuestPDF)



 Agent 6 : Email Composer (LLM)

Rôle : Rédige l'email personnalisé en utilisant le contexte final (offre analysée, compétences matchées, score obtenu).

Input : offre_json + match_result + profil_json

Output : email_json { objet, corps, signature }

▼
END → Retour JSON complet vers .NET

Agent de Notification (Optionnel — SignalR)

Agent de Notification

Rôle : Informe le frontend du succès de chaque étape via .NET SignalR (WebSocket).

À chaque fin d'agent, un message est envoyé :

- "Analyse de l'offre terminée ☒
- "Profil récupéré ☒
- "Score calculé : 78% ☒
- "CV prêt à être généré ☒
- "Email rédigé ☒

Implémenté via : .NET NotificationService + SignalR Hub

État partagé (AgentState)

```
class AgentState(TypedDict):  
    # --- Entrées ---  
    user_id: str  
    offer_text: str  
    template_id: str  
  
    # --- Sorties des agents ---  
    offre_json: dict          # ← Agent 1 (Analyseur)  
    profil_json: dict         # ← Agent 2 (Retriever)  
    normalized_data: dict     # ← Agent 3 (Normalisateur)  
    match_result: dict        # ← Agent 4 (Scorer)  
    cv_data_json: dict        # ← Agent 5 (Formatteur)  
    email_json: dict          # ← Agent 6 (Email)
```

Chronologie d'exécution

0.0s → Agent 1 (LLM – Analyse offre)	→ ~3.0s
3.0s → Agent 2 (RAG – Récupération profil)	→ ~3.1s
3.1s → Agent 3 (Algo – Normalisation)	→ ~3.15s
3.15s → Agent 4 (Calcul – Scoring)	→ ~3.20s
3.20s → Agent 5 (JSON – Formatage template)	→ ~3.30s
3.30s → Agent 6 (LLM – Rédaction email)	→ ~6.0s

TOTAL : ~6 secondes

5. 🎯 Les 11 fonctionnalités détaillées

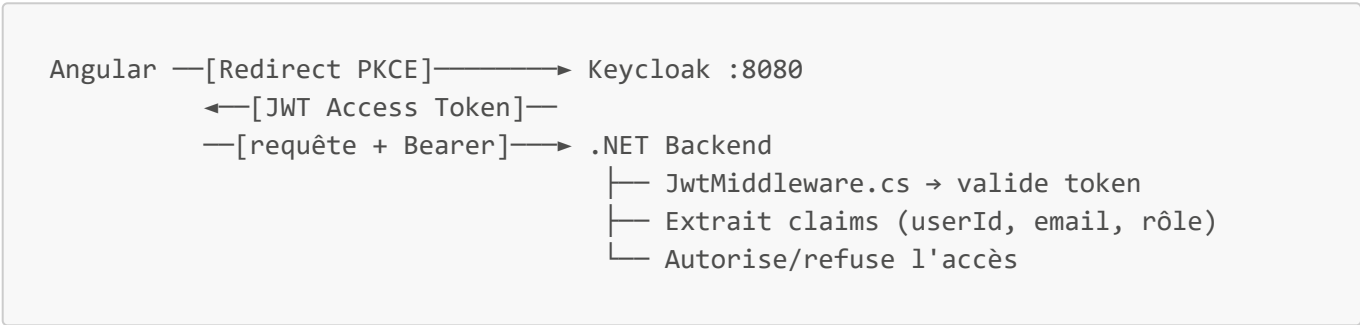
Fonctionnalité 1 : 🛡️ Authentification SSO (Keycloak)

Stack : `Keycloak 24` · `keycloak-js` · `Angular` · `.NET JwtMiddleware`

Ce que l'utilisateur voit :

- Page de login Keycloak (email/mot de passe, Google, etc.)
- Redirection automatique vers le dashboard après connexion
- Avatar + nom dans la navbar

Comment ça marche :



Configuration Keycloak :

- Realm : `nextstep`
- Client Angular : `nextstep-frontend` (Public, PKCE)
- Client .NET : `nextstep-backend` (Confidential)
- Rôles : `USER`, `ADMIN`

Fichiers :

Couche	Fichier	Rôle
Angular	<code>keycloak.service.ts</code>	Init Keycloak, login/logout, refresh token
Angular	<code>auth.guard.ts</code>	Protège les routes si non connecté
Angular	<code>auth.interceptor.ts</code>	Attache <code>Authorization: Bearer <JWT></code> automatiquement
.NET	<code>AuthController.cs</code>	Endpoint <code>/api/auth/me</code>
.NET	<code>KeycloakService.cs</code>	Validation/introspection JWT
.NET	<code>JwtMiddleware.cs</code>	Middleware pipeline ASP.NET

Fonctionnalité 2 : 👤 Profil Candidat

Stack : `.NET (EF Core)` · `Angular (Reactive Forms)` · `PostgreSQL`

Ce que l'utilisateur voit :

- Formulaire multi-sections avec onglets :
 -  Informations personnelles (nom, email, téléphone, ville)
 -  Compétences (tags : Angular, Python, Docker...)
 -  Expériences professionnelles (titre, entreprise, dates, description)
 -  Formations (diplôme, établissement, année)
 -  Certifications (nom, organisme, date)
 -  Projets académiques (titre, description, technologies)
- Bouton "Sauvegarder"

Comment ça marche :

```
Angular —POST /api/profile→ .NET ProfileController
                                |
                                |— Valide ProfileDto
                                |— ProfileService (logique métier)
                                |— ProfileRepository → EF Core → PostgreSQL
```

Tables PostgreSQL : *utilisateurs, profils, experiences, formations, certifications, competences, projets*

Fonctionnalité 3 : Analyse d'offre d'emploi (IA)

Stack : *Python (LangChain) · FastAPI · LLM (GPT/Groq)*

Ce que l'utilisateur voit :

- Un textarea pour coller le texte brut de l'offre
- Un sélecteur de template CV (Modern, Classic, Creative)
- Bouton "Analyser et Générer"
- Un loader animé pendant le traitement (~6 secondes)

Comment ça marche :

```
Angular —POST /api/offers/submit→ .NET OfferController
                                |
                                |— Sauvegarde offre brute en DB
                                |— POST http://agents-python:8000/run-
pipeline
                                |
                                | Pipeline LangGraph (6 agents)
                                |— Retourne JSON complet
```

Fonctionnalité 4 : Matching Profil ↔ Offre

Stack : *Python (algorithme pur) — PAS de LLM*

Ce que l'utilisateur voit :

- Badge circulaire avec le score de matching (ex: 72%)
- Liste verte des compétences retrouvées
- Liste rouge des compétences manquantes
- Recommandations textuelles

Géré par : Agent 3 (Normalisateur) + Agent 4 (Scorer) du pipeline.

Fonctionnalité 5 :  Score ATS (Algorithme déterministe)

Stack : Python (algorithme pur) — **PAS de LLM**

Ce que l'utilisateur voit :

- Score ATS 0-100 affiché à côté du CV
- Mots-clés présents/manquants
- Recommandations pour améliorer le score

Géré par : Agent 4 (Scorer) avec pondération positionnelle.

Fonctionnalité 6 :  Génération de CV Personnalisé (PDF)

Stack : Python (Formateur) · QuestPDF (.NET C#) · Templates pré-conçus

★ **Principe clé :** Le CV n'est **PAS** une copie du profil. Le système **SÉLECTIONNE** les éléments pertinents à l'offre, **IGNORE** les éléments non pertinents, et **RÉORDONNE** le reste pour **maximiser le score ATS**.

Ce que l'utilisateur voit :

- Aperçu PDF intégré dans la page
- CV personnalisé **sur mesure** pour l'offre ciblée
- Score ATS affiché à côté du CV
- Bouton "Télécharger PDF"

Flux :

```
Pipeline Python (Agent 5) produit cv_data_json
|
▼
.NET reçoit le JSON
|
▼
QuestPDF (CVGeneratorService.cs) → génère le PDF
|
▼
PDF sauvegardé + retourné au frontend
```

3 templates disponibles :

- 🎨 **Modern** — Design épuré, couleurs vives, icônes
- 📄 **Classic** — Professionnel traditionnel, noir et blanc
- ⚡ **Creative** — Layout original, sidebar colorée

Fonctionnalité 7 : ✉ Email de candidature personnalisé

Stack : Python (LangChain) · .NET (MailKit) · SMTP

Ce que l'utilisateur voit :

- Aperçu de l'email généré par l'IA (objet, corps, signature)
- Boutons "Modifier" et "Valider et Envoyer"
- ⚠ L'email n'est **JAMAIS** envoyé sans validation humaine

Comment ça marche :



Fonctionnalité 8 : 📋 Suivi des candidatures

Stack : .NET · Angular · Python (imaplib)

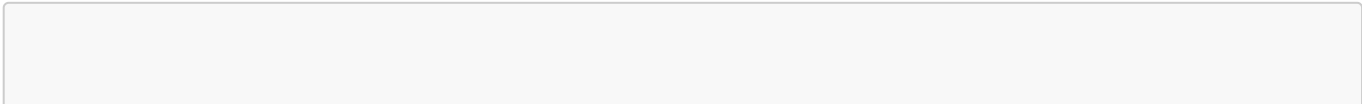
Ce que l'utilisateur voit :

- Tableau **Kanban** avec colonnes glissantes :
 - 📄 En attente → ✉ Envoyé → 👁 Vu → 📅 Entretien → ☑ Retenu → ❌ Refusé
- Chaque carte affiche : entreprise, poste, date d'envoi, score ATS

Fonctionnalité 9 : 🔄 Relance automatique (7 jours)

Stack : .NET (Hangfire) · Python (LangChain) · MailKit

Comment ça marche :



Hangfire (FollowUpJob.cs) – S'exécute toutes les 24 heures

```
|— SELECT * FROM candidatures
|   WHERE statut = 'ENVOYE'
|   AND date_envoi < NOW() - INTERVAL '7 days'
|   AND relance_envoyee = false
|
|— Pour chaque candidature trouvée :
|   |— POST /generate-email (type = "relance") → Python LangChain
|   |— Envoi email via MailKit
|   |— MAJ : relance_envoyee = true
|   |— Notification SignalR → Angular 🔔
```

Fonctionnalité 10 : 💬 Chatbot préparation entretien

Stack : Python (LangChain ConversationChain) · SignalR · WebSocket

Ce que l'utilisateur voit :

- Bulle de chat (en bas à droite, style ChatGPT)
- Questions d'entretien contextualisées (basées sur l'offre + profil)
- Feedback structuré (méthode STAR)

Fonctionnalité 11 : 🔔 Notifications temps réel

Stack : .NET (SignalR) · Angular

Types de notifications :

- ☒ "Votre CV a été généré avec succès"
- 🗨️ "Réponse reçue de l'entreprise OCP"
- 📧 "Relance automatique envoyée à Société XYZ"
- 💬 "Nouvelle question d'entretien disponible"

6. 📁 Structure des fichiers

Frontend — Angular 19

```
frontend/
|— Dockerfile
|— angular.json
|— package.json
|— tsconfig.json
|— src/
|   |— index.html
|   |— main.ts
|   |— styles.css
```

```

└─ app/
  └─ app.component.ts
  └─ app.routes.ts
  └─ core/
    └─ auth/
      └─ keycloak.service.ts
      └─ auth.guard.ts
      └─ auth.interceptor.ts
    └─ models/
      └─ user.model.ts
      └─ profile.model.ts
      └─ offer.model.ts
      └─ cv.model.ts
      └─ email.model.ts
      └─ candidature.model.ts
  └─ shared/components/
    └─ navbar/
    └─ sidebar/
    └─ card/
    └─ loader/
  └─ features/
    └─ auth/          (login, callback)
    └─ profile/       (personal-info, skills, experience, etc.)
    └─ offers/        (offer-paste, analysis-result)
    └─ cv/            (template-selector, cv-preview, ats-score)
    └─ email/         (email-preview, inbox-monitor)
    └─ candidature/   (candidature-tracker, followup-alert)
    └─ chatbot/       (chat-interface, chat-bubble)

```

Backend — .NET 8

```

backend/
└─ Dockerfile
└─ .env                ← Variables d'environnement (NE PAS COMMIT)
└─ .env.example        ← Template des variables
└─ NextStep.API/
  └─ Program.cs
  └─ appsettings.json
  └─ Modules/
    └─ Auth/
      └─ Controllers/  AuthController.cs
      └─ Services/     KeycloakService.cs
      └─ Middleware/   JwtMiddleware.cs
    └─ Profile/
      └─ Controllers/  ProfileController.cs
      └─ Services/     ProfileService.cs
      └─ Repositories/ ProfileRepository.cs
      └─ DTOs/         ProfileDto.cs
      └─ Models/       Profil.cs, Experience.cs, Formation.cs...
    └─ Offer/
      └─ Controllers/  OfferController.cs

```

```

├── Services/ OfferService.cs
├── Repositories/ OfferRepository.cs
├── CV/
│   ├── Controllers/ CVController.cs
│   ├── Services/ CVService.cs, CVGeneratorService.cs
│   └── Templates/ ModernTemplate.cs, ClassicTemplate.cs,
CreativeTemplate.cs
├── Email/
│   ├── Controllers/ EmailController.cs
│   └── Services/ EmailService.cs
├── Candidature/
│   ├── Controllers/ CandidatureController.cs
│   ├── Services/ CandidatureService.cs, FollowUpService.cs
│   └── Jobs/ FollowUpJob.cs (Hangfire)
├── Chatbot/
│   └── Hubs/ ChatHub.cs (SignalR)
├── Notification/
│   ├── Controllers/ NotificationController.cs
│   ├── Services/ NotificationService.cs
│   └── Hubs/ NotificationHub.cs
├── Infrastructure/
│   ├── Data/ AppDbContext.cs, Migrations/
│   └── Http/ AgentHttpClient.cs
├── Common/
│   ├── Middleware/ ExceptionMiddleware.cs
│   └── Extensions/ ServiceCollectionExtensions.cs

```

Agents — Python FastAPI + LangChain + LangGraph

```

agents/
├── Dockerfile
├── requirements.txt
├── .env          ← Variables d'environnement (NE PAS COMMIT)
├── .env.example  ← Template des variables
├── main.py       (monte tous les routers)
├── shared/
│   ├── config.py      (pydantic-settings → lit .env)
│   ├── database.py    (SQLAlchemy async → PostgreSQL + pgvector)
│   └── http_client.py  (httpx → appels vers .NET)
├── pipeline/
│   ├── state.py        (AgentState TypedDict)
│   ├── graph.py        (LangGraph StateGraph)
│   └── nodes.py         (6 fonctions d'agents)
├── ai_agent/
│   ├── router.py        (POST /analyze-offer, POST /match)
│   ├── service.py
│   ├── schemas.py       (OfferInput, ProfileJson, MatchResult)
│   └── agents/
│       ├── offer_analyzer.py  (🧠 Agent 1 – LLM)
│       ├── profile_retriever.py (📖 Agent 2 – RAG pgvector)
│       └── normalizer.py      (🧮 Agent 3 – Algorithme)

```



```
// Infrastructure/Http/AgentHttpClient.cs
public class AgentHttpClient
{
    private readonly HttpClient _http;

    public AgentHttpClient(HttpClient http)
    {
        _http = http;
        _http.BaseAddress = new Uri("http://agents-python:8000");
    }

    public async Task<PipelineResult> RunPipelineAsync(PipelineRequest request)
    {
        var response = await _http.PostAsJsonAsync("/run-pipeline", request);
        return await response.Content.ReadFromJsonAsync<PipelineResult>();
    }
}
```

8. 📄 Base de données — Schéma

Instance unique PostgreSQL (2 schémas)

```
nextstep_db (PostgreSQL 15 + pgvector)
├─ Schema: public          → Données applicatives
└─ Schema: keycloak_schema → Données Keycloak (géré automatiquement)
```

Tables principales (Schema **public**)

Table	Champs clés	Relations
utilisateurs	id, keycloak_id, email, nom, prenom	1:1 → profils
profils	id, utilisateur_id, titre, resume, telephone, ville	1:N → competences, experiences, formations, certifications, projets
competences	id, profil_id, nom, niveau	N:1 → profils
experiences	id, profil_id, titre, entreprise, date_debut, date_fin, description	N:1 → profils
formations	id, profil_id, diplome, etablissement, annee	N:1 → profils
certifications	id, profil_id, nom, organisme, date	N:1 → profils
projets	id, profil_id, titre, description, technologies	N:1 → profils

Table	Champs clés	Relations
offres_emploi	id, utilisateur_id, texte_brut, analyse_json (JSONB), date_creation	N:1 → utilisateurs
cv_generes	id, utilisateur_id, offre_id, template_id, pdf_path, score_ats, date_creation	N:1 → utilisateurs, offres
candidatures	id, utilisateur_id, offre_id, cv_id, statut, date_envoi, relance_envoyee	N:1 → utilisateurs
historique_statuts	id, candidature_id, etat_statut, date_mise_a_jour, commentaire	N:1 → candidatures
emails	id, candidature_id, objet, corps, signature, type, date_envoi	N:1 → candidatures
notifications	id, utilisateur_id, type, message, lu, date_creation	N:1 → utilisateurs

9. 🗎 Endpoints API complets

Backend .NET (/api/*)

Méthode	Endpoint	Module	Description
POST	/api/auth/me	Auth	Infos utilisateur connecté
GET	/api/profile	Profile	Lire le profil complet
POST	/api/profile	Profile	Créer le profil
PUT	/api/profile	Profile	Mettre à jour le profil
POST	/api/offers/submit	Offer	Soumettre offre + lancer pipeline
GET	/api/offers/:id/analysis	Offer	Résultat analyse IA
POST	/api/cv/generate	CV	Lancer génération PDF
GET	/api/cv/:id/download	CV	Télécharger le PDF
POST	/api/emails/send	Email	Envoyer email (après validation)
GET	/api/candidatures	Candidature	Liste des candidatures
PUT	/api/candidatures/:id/status	Candidature	MAJ statut
GET	/api/notifications	Notification	Liste notifications
PUT	/api/notifications/:id/read	Notification	Marquer comme lu

Agents Python (<http://agents-python:8000>)

Méthode	Endpoint	Service	Description
---------	----------	---------	-------------

Méthode	Endpoint	Service	Description
POST	/run-pipeline	pipeline	Lance tout le pipeline multi-agent (6 agents)
POST	/analyze-offer	ai_agent	Analyse LLM de l'offre → JSON
POST	/match	ai_agent	Score matching profil ↔ offre
POST	/prepare-cv-data	cv_engine	Prépare JSON CV pour QuestPDF
POST	/generate-email	email_engine	Génère email candidature (ou relance)
GET	/inbox	email_engine	Statut boîte mail (IMAP)
WS	/ws/chat	chatbot	WebSocket chatbot entretien
GET	/health	—	Health check

10. 🛠️ Infrastructure & Docker

Docker Compose — Vue d'ensemble

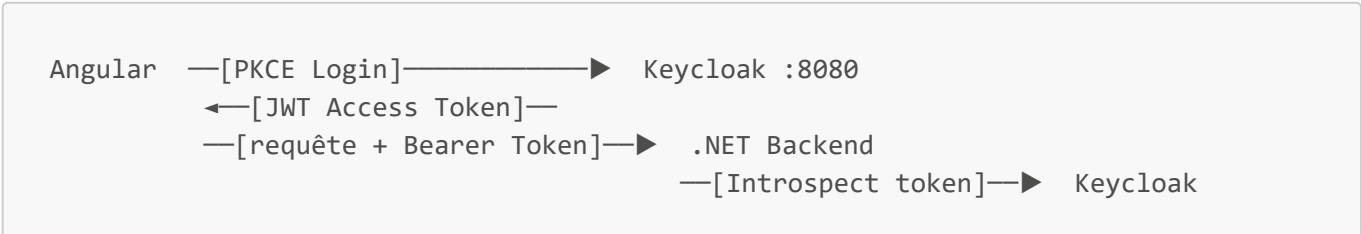
Service	Image	Port	Rôle
postgres	pgvector/pgvector:15-pg11	5432	BDD unique (app + Keycloak) avec support Vector Search
keycloak	quay.io/keycloak/keycloak:24.0	8080	Authentification SSO
redis	redis:7-alpine	6379	Cache + Hangfire background jobs
backend	./backend/Dockerfile	5000	API .NET 8 (Monolithe modulaire)
agents-python	./agents/Dockerfile	8000	Pipeline IA (LangGraph + FastAPI)
frontend	./frontend/Dockerfile	4200	Angular 19 SPA

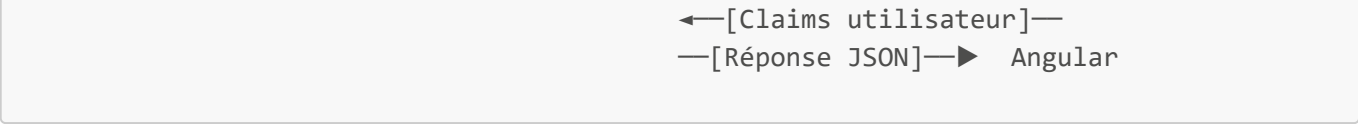
Ordre de démarrage

```
postgres (healthcheck) → keycloak → redis → backend → agents-python → frontend
```

11. 🛡️ Sécurité & Authentification

Flux Keycloak complet





Détails

Élément	Valeur
Realm	nextstep
Client Angular	nextstep-frontend (Public, PKCE)
Client .NET	nextstep-backend (Confidential)
Rôles	USER, ADMIN
Token	JWT RS256, 5 min access, 30 min refresh
Interceptor	auth.interceptor.ts → inject Bearer auto

12. 🔑 Variables d'environnement

Backend (.NET) — backend/.env

```
# Database
ConnectionStrings__DefaultConnection=Host=postgres;Database=nextstep_db;Username=admin;Password=password

# Keycloak
Keycloak__Authority=http://keycloak:8080/realms/nextstep
Keycloak__ClientId=nextstep-backend
Keycloak__ClientSecret=VOTRE_SECRET_ICI

# Redis
Redis__Configuration=redis:6379

# Python Agents
PythonAgents__Url=http://agents-python:8000

# SMTP (Email)
Smtp__Host=smtp.gmail.com
Smtp__Port=587
Smtp__Username=votre-email@gmail.com
Smtp__Password=votre-app-password
Smtp__FromName=NextStep
```

Agents Python — agents/.env

```
# Database
DATABASE_URL=postgresql+asyncpg://admin:password@postgres:5432/nextstep_db
```

```
# LLM Provider
OPENAI_API_KEY=sk-xxxxxxxxxxxxxxxxxxxxxx
# OU
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxxxx

# Keycloak
KEYCLOAK_URL=http://keycloak:8080

# Backend .NET
BACKEND_URL=http://backend:5000

# IMAP (Email Monitoring)
IMAP_HOST=imap.gmail.com
IMAP_PORT=993
IMAP_USERNAME=votre-email@gmail.com
IMAP_PASSWORD=votre-app-password
```

13. 👥 Équipe & Répartition

Membre	Module	Stack	Responsabilité
M1	Auth + Profile	.NET + Angular	Keycloak, Utilisateur, Profil complet
M2	AI Agent	Python FastAPI	Agents 1-4 : Analyse offre, RAG, normalisation, scoring
M3	CV Engine	.NET (QuestPDF) + Python	Agent 5 : Formatage template, génération PDF, 3 templates
M4	Email Engine	.NET + Python	Agent 6 : Email LangChain, SMTP MailKit, IMAP, relance 7j
M5	Chatbot + Frontend	Python + Angular	UI complète, chatbot LangChain, notifications SignalR

Comment collaborer efficacement

1. Chaque membre travaille sur son MODULE uniquement
 2. Les contrats d'API (DTOs / Pydantic schemas) sont définis EN PREMIER
 3. Chaque module est INDÉPENDANT :
 - Son propre dossier (Controllers/, Services/, etc.)
 - Ses propres DTOs
 - Ses propres tests
 4. L'intégration se fait via les ENDPOINTS définis
 5. Le docker-compose assemble le tout

NextStep — Référence Complète — v6.0 — Mars 2026 Angular 19 · .NET 8 · Python 3.12 · FastAPI · LangChain · LangGraph · QuestPDF · Keycloak · PostgreSQL (pgvector) · SignalR · MailKit · Redis · Hangfire · Docker